

Prestandajämförelse av React och Vanilla JavaScript för webbapplikationer

Performance Comparison of React and Vanilla JavaScript for Web Applications

Examensarbete inom huvudområdet
Informationsteknologi
Grundnivå 30 högskolepoäng
Vårtermin 2023

Jesper Jansson A23jesja

Handledare: Johan Bjurén
Examinator: Henrik Gustavsson

Sammanfattning

Detta examensarbete jämför prestanda mellan React och Vanilla JavaScript för en produktkatalog-applikation med 500 produkter. Syftet var att ge webbutvecklare konkreta prestandadata för att kunna göra välgrundade tekniska val. Samma applikation implementerades två gånger, en gång i Vanilla JavaScript och en gång i React, med identisk funktionalitet för sökning, filtrering och sortering. Prestanda mättes med Google Lighthouse med fokus på Core Web Vitals genom 20 tester (10 per version). Resultaten analyserades statistiskt med ANOVA-test. Resultaten visade att Vanilla JavaScript presterade statistiskt signifikant bättre än React på samtliga fem mätta prestandamått ($p < 0.05$). Vanilla JavaScript uppnådde perfekt Performance Score (100/100) medan React varierade mellan 94-95. Den största skillnaden observerades i LCP (Largest Contentful Paint) där Vanilla JavaScript laddade på 0.30 sekunder jämfört med Reacts 1.46 sekunder. Studien visar att för produktkatalog-applikationer där prestanda är kritiskt kan Vanilla JavaScript vara ett bättre val än React.

Nyckelord: React, Vanilla JavaScript, webbprestanda, Core Web Vitals, frontend-utveckling

Abstract

This thesis compares the performance of React and Vanilla JavaScript for a product catalog application with 500 products. The purpose was to provide web developers with concrete performance data to make informed technical decisions. The same application was implemented twice, once in Vanilla JavaScript and once in React, with identical functionality for search, filtering, and sorting. Performance was measured using Google Lighthouse with focus on Core Web Vitals through 20 tests (10 per version). Results were analyzed statistically using ANOVA tests. Results showed that Vanilla JavaScript performed statistically significantly better than React across all five measured performance metrics ($p < 0.05$). Vanilla JavaScript achieved perfect Performance Score (100/100) while React varied between 94-95. The largest difference was observed in LCP (Largest Contentful Paint) where Vanilla JavaScript loaded in 0.30 seconds compared to React's 1.46 seconds. The study demonstrates that for product catalog applications where performance is critical, Vanilla JavaScript may be a better choice than React.

Keywords: React, Vanilla JavaScript, web performance, Core Web Vitals, frontend development

Innehållsförteckning

1	Introduktion.....	1
2	Bakgrund.....	2
2.1	React och Virtual DOM.....	2
2.2	Vanilla JavaScript.....	3
2.3	Core Web Vitals.....	3
2.4	Tidigare forskning.....	4
3	Problemformulering.....	5
3.1	Metodbeskrivning.....	5
4	Metoddiskussion.....	7
4.1	Val av metod.....	7
4.2	Begränsningar.....	7
4.3	Alternativa metoder.....	8
5	Genomförande.....	9
5.1	Utvecklingsmiljö och verktyg.....	9
5.2	Generering av testdata.....	9
5.3	Implementation av Vanilla JavaScript-version.....	10
5.4	Implementation av React-version.....	11
5.5	Testmetodik och genomförande.....	12
5.6	Dataanalys.....	13
5.7	Litteraturstudie.....	14
6	Utvärdering.....	15
6.1.1	Visuell presentation av testresultat.....	15
6.1.2	Presentation av undersökning.....	17
	Vanilla JavaScript-resultat.....	18
	React-resultat.....	19
6.2	Analys.....	20
6.3	Slutsatser.....	20
6.3.1	Vad pilotstudien har lärt oss.....	21
6.3.2	Begränsningar i pilotstudien.....	21
6.3.3	Parametrar att justera för huvudstudien.....	21
6.3.4	Sammanfattning.....	21
7	Avslutande diskussion.....	22
7.1	Diskussion.....	22
7.1.1	Tolkning av resultat.....	22
7.1.2	Jämförelse med tidigare forskning.....	22
7.1.3	Praktiska implikationer.....	23
7.1.4	Begränsningar och validitet.....	23
7.2	Slutsats.....	24
7.3	Etiska aspekter.....	24
7.4	Samhällelig nytta.....	24
7.5	Framtida arbete.....	25
8	Referenser.....	26

1 Introduktion

Webbutveckling har förändrats mycket de senaste åren. Idag bygger man inte bara enkla statiska webbsidor, utan ofta komplexa och interaktiva webbapplikationer. En fråga som många webbutvecklare står inför är om man ska använda ett JavaScript-ramverk som React eller bygga sin applikation med vanlig JavaScript, så kallat Vanilla JavaScript.

React utvecklades av Facebook och släpptes 2013. Det har blivit väldigt populärt och används idag av stora företag som Netflix och Instagram. React påstås vara mer effektivt än vanlig JavaScript tack vare något som heter Virtual DOM. Men samtidigt finns det diskussioner om huruvida det extra lagret som React innebär alltid är värt det, eller om Vanilla JavaScript ibland kan vara snabbare och enklare.

Prestanda är väldigt viktigt för webbapplikationer. Google har infört Core Web Vitals för att mäta hur bra en webbsida är för användarna. Dessa mått påverkar både hur användare upplever sidan och hur den rankas i sökmotorer. Om en webbplats är för långsam tappar man användare.

Även om React är populärt saknas det tydliga riktlinjer för när React faktiskt ger bättre prestanda än Vanilla JavaScript. Många utvecklare verkar välja React automatiskt utan att testa om det är snabbast för just deras projekt. Därför vill jag göra en systematisk jämförelse mellan React och Vanilla JavaScript för en specifik typ av webbapplikation - en produktkatalog med 500 produkter. Jag kommer att bygga samma applikation två gånger och sedan mäta prestanda skillnaderna

2 Bakgrund

2.1 React och Virtual DOM

React är ett JavaScript-bibliotek för att bygga användargränssnitt. Det utvecklades av Facebook och släpptes som öppen källkod 2013. React bygger på en komponentbaserad arkitektur, vilket betyder att man delar upp sitt användargränssnitt i små, återanvändbara delar. Idag används React av stora företag som Netflix, Instagram och Airbnb (Siahaan & Vianto 2022).

Det som gör React unikt är Virtual DOM. När något ändras i applikationen uppdaterar inte React direkt webbläsarens DOM, utan skapar först en virtuell version i minnet. Sedan jämför React den gamla och nya virtuella versionen genom en process kallad "diffing" och uppdaterar bara det som faktiskt har ändrats i den riktiga DOM (Cechinel, Laranjeira & Figueiredo 2023). Detta illustreras i följande kodexempel:

```
function ProductList({ products }) {  
  return (  
    <div>  
      {products.map(product => (  
        <ProductCard key={product.id} product={product} />  
      ))}  
    </div>  
  );  
}
```

Exempel 1: React-komponent som renderar produkter

När tillståndet ändras, till exempel när användaren filtrerar produkter, skapar React en ny Virtual DOM och jämför den med den gamla. Endast de produktkort som faktiskt ändrats uppdateras i den riktiga DOM, vilket ska göra uppdateringar snabbare (Cechinel, Laranjeira & Figueiredo 2023).

Cechinel, Laranjeira och Figueiredo (2023) jämförde React och Vue.js i en studie där de tittade på minnesanvändning, CPU-användning och renderingstid. De kom fram till att olika designmönster kan påverka prestandan mycket, och att ramverkens overhead kan variera beroende på hur man implementerar dem. Deras studie visade att React ofta kräver mer minne än enklare alternativ, men kan vara snabbare vid komplexa uppdateringar.

2.2 Vanilla JavaScript

Vanilla JavaScript är helt enkelt vanlig JavaScript utan några ramverk eller bibliotek. Namnet används för att skilja på grundläggande JavaScript från alla ramverk som finns (Kaluža, Trošelj & Vukelić 2018). När man använder Vanilla JavaScript manipulerar man DOM direkt genom webbläsarens inbyggda funktioner.

Samma funktionalitet som React-exemplet ovan kan implementeras i Vanilla JavaScript så här:

```
function renderProducts(products) {
  const container = document.getElementById('products');
  container.innerHTML = "";

  products.forEach(product => {
    const card = document.createElement('div');
    card.className = 'product-card';
    card.innerHTML = `<h3>${product.name}</h3>`;
    container.appendChild(card);
  });
}
```

Exempel 2: Vanilla JavaScript som renderar produkter

Fördelen med Vanilla JavaScript är att man inte behöver ladda in något extra ramverk. Det ger mindre filstorlek och potentiellt snabbare laddningstid. Andersson (2020) fann i sin studie att Vanilla JavaScript-applikationer var betydligt mindre i filstorlek jämfört med ramverk-baserade alternativ. Nackdelen är att man själv måste skriva mer kod för att hantera uppdateringar och hålla koll på vad som ändras.

Andersson (2020) gjorde en omfattande studie där han jämförde DOM-prestanda mellan Vanilla JavaScript och ramverk som Angular, React och Vue.js. Han fann att alla alternativ kunde leverera bra prestanda i moderna webbläsare, men att Vanilla JavaScript hade en betydande fördel när det gäller initial laddningstid och minnesanvändning på grund av avsaknaden av ramverksoverhead.

2.3 Core Web Vitals

Google introducerade Core Web Vitals 2020 som ett sätt att mäta användarupplevelse på webben (Google Developers 2020). Core Web Vitals består av tre huvudmått:

- Largest Contentful Paint (LCP) - mäter laddningstid. För bra användarupplevelse bör LCP vara under 2,5 sekunder. LCP mäter när det största synliga elementet på sidan har renderats.
- Interaction to Next Paint (INP) - mäter hur snabbt sidan reagerar när man klickar. Bör vara under 200 millisekunder. INP ersatte First Input Delay (FID) som standardmått 2024.
- Cumulative Layout Shift (CLS) - mäter hur mycket innehållet "hoppas runt" när sidan laddas. Bör vara under 0,1. CLS mäter visuell stabilitet under laddning.

Dessa mått är viktiga inte bara för användarupplevelsen, utan påverkar också hur Google rankar webbsidor i sökresultaten (Google Developers 2020). Van Riet et al. (2023) genomförde en industristudie där de förbättrade prestandan på en webbapplikation genom olika optimeringar. De fann en stark koppling mellan objektiva mätningar som Core Web Vitals och hur användare subjektivt upplever laddningstiden. Deras studie visade att även små förbättringar i Core Web Vitals kan ha märkbar påverkan på användarupplevelsen.

2.4 Tidigare forskning

Selakovic och Pradel (2016) gjorde en omfattande empirisk studie av prestandaproblem i JavaScript där de analyserade 10 000 webbplatser. De kom fram till att ineffektiv användning av API:er var den vanligaste orsaken till dålig prestanda, och att många optimeringar är relativt enkla att implementera men att utvecklare ofta inte vet vilka metoder som är mest effektiva. Deras studie visade att 42% av alla optimeringar förbättrar prestanda konsekvent över olika JavaScript-motorer.

Biørn-Hansen et al. (2020) genomförde en empirisk undersökning av prestandaoverhead i cross-platform ramverk för mobilutveckling. Även om deras fokus var på mobilappar, fann de att ramverksoverhead kan ha signifikant påverkan på prestanda, särskilt för resurskrävande operationer. Denna forskning är relevant för webbutveckling eftersom många moderna webbramverk delar liknande arkitektoniska utmaningar.

Siahaan och Vianto (2022) genomförde en komparativ studie av frontend JavaScript-ramverk med fokus på prestanda mätt med Google Lighthouse. Deras studie jämförde React, Vue och Angular och fann att olika ramverk presterar olika beroende på applikationstyp och användningsfall. De betonade vikten av att välja rätt ramverk baserat på specifika krav snarare än allmän popularitet.

Malavolta et al. (2017) undersökte hur Service Workers påverkar energiförbrukningen i Progressive Web Apps. De fann att dessa tekniker kan påverka batteriförbrukningen mycket i mobila enheter. Studien visar att prestandaoptimering handlar om mer än bara hastighet - det handlar också om hur mycket resurser som används, vilket blir alltmer kritiskt med ökad mobil användning.

Oliveira et al. (2023) analyserade resursanvändningen för olika mobilappsutvecklingsramverk och fann att Flutter ofta hade minst overhead medan React Native hade högst overhead i de flesta benchmarks. Även om studien fokuserade på mobilappar, är resultaten relevanta eftersom React för webben delar många arkitektoniska egenskaper med React Native.

Trots dessa studier saknas det systematiska jämförelser mellan React och Vanilla JavaScript för samma typ av webbapplikation med fokus på moderna mätningar som Core Web Vitals. Det är det gapet jag vill försöka fylla med det här arbetet.

3 Problemformulering

När man ska bygga en webbapplikation står man ofta inför valet mellan att använda React eller Vanilla JavaScript. React marknadsförs som mer effektivt tack vare Virtual DOM (Cechinel, Laranjeira & Figueiredo 2023), men det betyder också att man måste ladda in extra kod från React-biblioteket. Som Andersson (2020) visade i sin studie var Vanilla JavaScript-applikationer betydligt mindre i filstorlek, vilket kan ge snabbare initial laddning.

Problemet är att många utvecklare verkar välja React som standard utan att undersöka om det faktiskt ger bättre prestanda för just deras projekt. Detta bekräftas av Selakovic och Pradel (2016) som fann att ineffektiv användning av API:er och ramverk är den vanligaste orsaken till prestandaproblem i JavaScript-applikationer. Samtidigt saknas det vetenskapliga jämförelser som mäter prestanda med moderna standarder som Core Web Vitals (Google Developers 2020).

Van Riet et al. (2023) visade i sin industristudie att det finns en stark koppling mellan objektiva prestandamätningar och användarupplevelse. Detta gör det viktigt att förstå hur olika tekniska val påverkar mätbara prestandavärden. Biørn-Hansen et al. (2020) fann också att ramverksoverhead kan ha signifikant påverkan på prestanda, vilket gör det relevant att undersöka om React's Virtual DOM-overhead är värt det för alla typer av applikationer.

Därför vill jag undersöka följande forskningsfråga:

Hur påverkar valet mellan React och Vanilla JavaScript prestandan för en produktkatalog-applikation med 500 produkter, mätt genom Core Web Vitals?

För att kunna svara på frågan har jag formulerat följande hypoteser:

H1 (Hypotes): Vanilla JavaScript kommer att ge bättre prestanda än React för en produktkatalog med 500 produkter, mätt genom Core Web Vitals (LCP, INP, CLS), filstorlek och minnesanvändning.

H0 (Nollhypotes): Det finns ingen statistiskt signifikant skillnad i prestanda mellan Vanilla JavaScript och React för den här typen av applikation.

3.1 Metodbeskrivning

Jag kommer använda en experimentell metod där jag bygger samma webbapplikation två gånger - en gång med Vanilla JavaScript och en gång med React. Detta tillvägagångssätt har använts i tidigare studier av JavaScript-ramverk (Cechinel, Laranjeira & Figueiredo 2023; Andersson 2020; Siahaan & Vianto 2022) och gör att jag kan jämföra prestanda direkt mellan de två versionerna under samma förutsättningar.

Applikationen kommer att vara en produktkatalog med 500 produkter som har funktioner för sökning, filtrering och sortering. Jag kommer att mäta prestanda med Google Lighthouse och Chrome DevTools, med fokus på Core Web Vitals (Google Developers 2020). Lighthouse är branschstandard för webbprestandamätning och har använts i liknande studier (Siahaan & Vianto 2022; Van Riet et al. 2023).

Varje test kommer upprepas 20 gånger för varje version för att få tillförlitliga resultat. Enligt Selakovic och Pradel (2016) är det viktigt att göra upprepade mätningar eftersom JavaScript-prestanda kan variera mellan körningar. Därefter analyserar jag datan med ANOVA-test för att se om skillnaderna är statistiskt signifikanta ($p < 0.05$).

4 Metoddiskussion

4.1 Val av metod

Jag valde en experimentell metod för att jag då kan jämföra React och Vanilla JavaScript under kontrollerade förhållanden. Genom att bygga exakt samma app med båda teknikerna kan jag vara säker på att eventuella skillnader i prestanda beror på själva tekniken och inte på andra faktorer. Det här sättet att jämföra har använts i tidigare studier av JavaScript-ramverk (Cechinel, Laranjeira & Figueiredo 2023; Andersson 2020; Siahaan & Vianto 2022) och har visat sig ge tillförlitliga resultat.

Google Lighthouse valdes som verktyg eftersom det är branschstandard för webbprestanda och mäter direkt Core Web Vitals (Google Developers 2020). Siahaan och Vianto (2022) använde Lighthouse i sin jämförelse av JavaScript-ramverk och fann att det ger konsekventa och reproducerbara resultat. Chrome DevTools används också för att få mer detaljerad information om interaktioner som sökning och filtrering. Båda verktygen är gratis och välkända, vilket gör det lättare för andra att upprepa min studie.

Valet av 500 produkter baseras på att det är tillräckligt många för att testa prestanda under realistisk last, men inte så många att det blir omöjligt att hantera manuellt. Cechinel, Laranjeira och Figueiredo (2023) använde liknande storlekar i sina benchmarks för att simulera verkliga användningsfall.

4.2 Begränsningar

Det finns flera begränsningar med studien som är viktiga att tänka på:

- Applikationstyp: Resultaten gäller bara för produktkataloger med 500 produkter. Som Siahaan och Vianto (2022) påpekade presterar olika ramverk olika beroende på applikationstyp. Andra typer av webbapplikationer (till exempel sociala medier, dashboards eller formulär) kan ge helt andra resultat.
- Skalning: Jag testar bara med 500 produkter. Det kan vara annorlunda med 50 eller 5000 produkter.
- Testmiljö: Testerna görs i en kontrollerad miljö. Som Biørn-Hansen et al. (2020) noterade kan riktiga användare ha olika datorer, internetuppkopplingar och webbläsare, vilket kan påverka prestandan annorlunda.
- Kodkvalitet: Resultaten förutsätter att jag kodar båda versionerna på rätt sätt. Selakovic och Pradel (2016) visade att ineffektiv API-användning är den vanligaste orsaken till prestandaproblem, så om jag gör dålig kod i någon av versionerna blir jämförelsen orättvis.
- Tidsbegränsning: Som examensarbete har jag begränsat med tid, vilket påverkar hur omfattande testerna kan bli.

4.3 Alternativa metoder

Jag funderade på flera andra sätt att göra studien:

- Jämföra fler ramverk: Att inkludera Vue.js, Angular eller Svelte hade gett en bredare bild, som i studien av Siahaan och Vianto (2022) som jämförde tre ramverk. Men det hade blivit för stort för ett examensarbete. Det hade krävt att bygga applikationen flera gånger till och göra många fler tester.
- Användarstudie: Att låta riktiga användare testa applikationerna hade gett värdefull insikt om upplevd prestanda, som Van Riet et al. (2023) gjorde i sin industristudie. Men det kräver etikprövning och mer tid. Det hade också varit svårt att kontrollera för andra faktorer som påverkar upplevelsen.
- Olika produktantal: Att testa med olika antal produkter (till exempel 100, 500 och 1000) hade visat hur prestanda skalas, men det hade tagit för lång tid.

Trots begränsningarna tycker jag att den valda metoden är bra för att svara på min forskningsfråga inom ramen för ett examensarbete. Den ger konkreta, mätbara resultat som kan jämföras statistiskt, vilket är i linje med tidigare forskning i området (Andersson 2020; Cechinel, Laranjeira & Figueiredo 2023; Siahaan & Vianto 2022).

5 Genomförande

Utvecklingsprocessen dokumenterades genom 20 commits i det publika GitHub-repositoryt (<https://github.com/a23jesja/react-vs-vanilla-prestanda>). Vanilla JavaScript-versionen implementerades i commits 3-8, där grundläggande funktionalitet (commit 3-4), sökfunktion (commit 6), kategorifiltrering (commit 7) och sortering (commit 8) dokumenterades separat. React-versionen byggdes i commits 9-17 med motsvarande progression. Testning, dataanalys och dokumentation genomfördes i commits 18-20.

5.1 Utvecklingsmiljö och verktyg

För att kunna utveckla och testa båda versionerna användes följande verktyg och miljöer:

Visual Studio Code användes som primär kodredigerare. Det gav stöd för både JavaScript och React-utveckling med syntax-highlighting och autokomplettering.

Git och GitHub användes för versionshantering. Alla ändringar dokumenterades genom regelbundna commits vilket möjliggjorde spårning av utvecklingsprocessen. Projektets repository finns på <https://github.com/a23jesja/react-vs-vanilla-prestanda> och innehöll vid testningens genomförande 20 commits.

Node.js (version 18+) användes för att köra JavaScript-skript och för att installera nödvändiga verktyg. För React-versionen krävdes Node.js för att kunna använda Vite.

Vite (version 8.0.5) användes som byggverktyg och utvecklingsserver för React-applikationen. Vite valdes för att det är snabbt och modernt, samt att det är rekommenderat av React-teamet.

Google Chrome och Chrome DevTools användes för all prestandamätning. Specifikt användes verktyget Lighthouse som är inbyggt i DevTools för att mäta Core Web Vitals och andra viktiga prestandamått.

5.2 Generering av testdata

Innan utvecklingen av själva applikationerna påbörjades behövdes testdata. Ett Node.js-skript skapades för att generera 500 slumpmässiga produkter. Nedan visas ett utdrag från genereringsskriptet:

```
const categories = ['Elektronik', 'Kläder',  
  'Hem & Trädgård', 'Sport', 'Leksaker'];  
  
for (let i = 1; i <= 500; i++) {  
  const category = categories[  
    Math.floor((i - 1) / 100)  
  ];  
  const pris = Math.floor(Math.random()  
    * 9900) + 100;  
  // ... skapar produkt }
```

Produkterna delades upp jämnt mellan de fem kategorierna (100 produkter per kategori) genom att använda $\text{Math.floor}((i - 1) / 100)$. Priset genererades slumpmässigt mellan 100-10000 kr. Datan sparades som en JSON-fil som kunde användas av båda versionerna.

5.3 Implementation av Vanilla JavaScript-version

Vanilla JavaScript-versionen utvecklades först eftersom den är enklare och kräver färre dependencies. Strukturen bestod av tre huvudfiler: *index.html*, *style.css* och *app.js*.

HTML-struktur

HTML-filen innehöll en enkel struktur med sökfält, filter-dropdowns och en container för produkterna:

```
<div class="controls">
  <input type="text"
    id="searchInput"
    placeholder="Sök produkter...">
  <select id="categoryFilter">
    <option value="">Alla kategorier</option>
  </select>
</div>
<div id="productGrid"></div>
```

JavaScript-logik

Produkterna laddades med *fetch()* och renderades dynamiskt:

```
fetch('../testdata/products.json')
  .then(response => response.json())
  .then(data => {
    allaProdukter = data;
    visaProdukter(allaProdukter);
  });
```

Sökfunktionen filtrerade produkter baserat på användarens input:

```
searchInput.addEventListener('input', () => {
  const searchTerm =
    searchInput.value.toLowerCase();
  const filtrerade = allaProdukter.filter(p =>
    p.namn.toLowerCase().includes(searchTerm)
  );
  visaProdukter(filtrerade);
});
```

Funktionaliteten implementerades steg för steg och varje steg dokumenterades genom separata Git-commits (commit 3-8).

5.4 Implementation av React-version

React-versionen skapades med Vite:

```
“ npm create vite@latest react-app
```

```
cd react-app
```

```
npm install “
```

Huvudkomponenten *App.jsx* byggdes upp med React Hooks. State hanterades med *useState*:

```
const [allaProdukter, setAllaProdukter] =  
  useState([]);  
const [visadeProdukter, setVisadeProdukter] =  
  useState([]);  
const [soktext, setSoktext] = useState("");
```

Produkter laddades när komponenten monterades:

```
useEffect(() => {  
  fetch('/data/products.json')  
    .then(response => response.json())  
    .then(data => {  
      setAllaProdukter(data);  
      setVisadeProdukter(data);  
    });  
}, []);
```

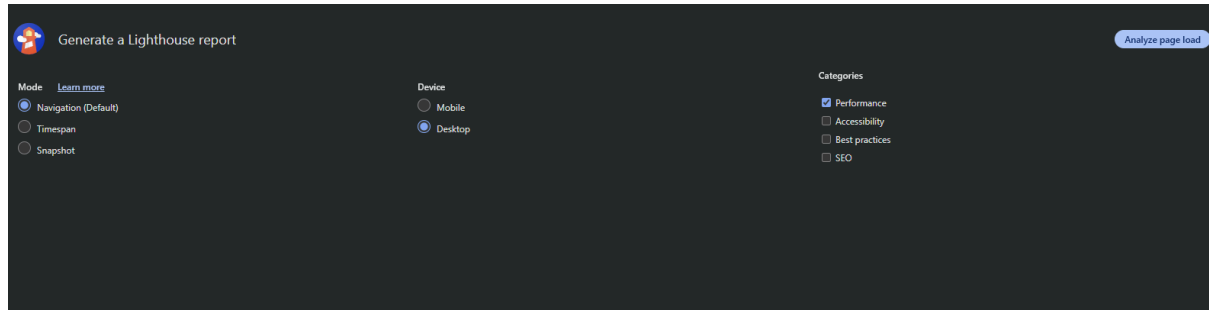
Filtrering och sortering hanterades i en separat *useEffect* som kördes när state ändrades:

```
useEffect(() => {  
  let filtrerade = allaProdukter;  
  if (soktext !== "") {  
    filtrerade = filtrerade.filter(p =>  
      p.namn.toLowerCase()  
        .includes(soktext.toLowerCase())  
    );  
  }  
  setVisadeProdukter(filtrerade);  
}, [soktext, allaProdukter]);
```

CSS:en kopierades från Vanilla JavaScript-versionen för att säkerställa identiskt utseende. Funktionaliteten implementerades i samma ordning (commit 9-17).

5.5 Testmetodik och genomförande

När båda versionerna var färdiga genomfördes prestandatestning med Google Lighthouse. Lighthouse är ett officiellt verktyg från Google för att mäta webbprestanda och Core Web Vitals.



Testuppsättning:

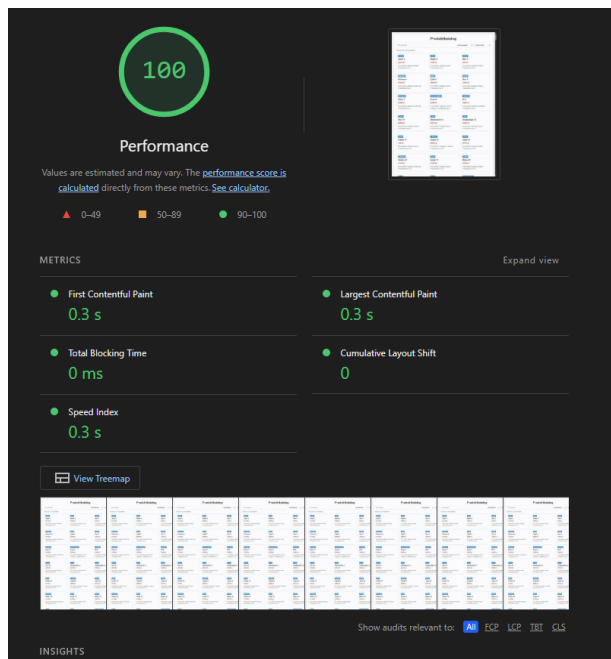
- Verktyg: Google Lighthouse i Chrome DevTools
- Enhet: Desktop-läge
- Kategori: Endast Performance
- Antal tester: 10 per version (totalt 20 tester)
- Mätta värden: Performance Score (0-100), LCP (sekunder), TBT (millisekunder), CLS (nummer), Speed Index (sekunder)

Testprotokoll:

För att säkerställa konsekventa resultat följdes ett strikt testprotokoll:

1. Chrome stängdes helt mellan varje test
2. Servern startades om (VS Code Live Server för Vanilla JS på port 5500, Vite dev server för React på port 5173)
3. Sidan öppnades i Chrome
4. DevTools öppnades (F12)
5. Lighthouse-fliken valdes
6. Inställningar kontrollerades (Desktop, Performance)
7. 'Analyze page load' klickades
8. Resultaten dokumenterades i Excel

Testerna kördes under samma förhållanden: datorn var inkopplad (inte på batteri), andra program var stängda och endast en Chrome-flik var öppen åt gången. Detta för att minimera externa faktorer som kunde påverka resultaten.



5.6 Dataanalys

Alla testresultat dokumenterades i en Excel-fil (*Lighthouse_Resultat.xlsx*). För varje test sparades de fem mätta värdena i separata kolumner.

När alla 20 tester var genomförda beräknades medelvärden för varje mått. En sammanfattande tabell skapades (*Resultat_Sammanfattning.xlsx*) som visade medelvärden, skillnader mellan versionerna och p-värden från statistisk analys.

För att statistiskt validera skillnaderna genomfördes ANOVA-test (Analysis of Variance). Ett Python-skript skapades som läste in testdatan och beräknade F-statistik och p-värden:

```
import scipy.stats as stats
import numpy as np

vanilla_performance = [100, 100, ...]
react_performance = [95, 95, 94, ...]

f_stat, p_value = stats.f_oneway(
    vanilla_performance,
    react_performance
)
```

ANOVA-testet använder scipy.stats-biblioteket och jämför variansen mellan de två grupperna. P-värden under 0.05 indikerar statistiskt signifikanta skillnader, vilket innebär att skillnaden är för stor för att vara en slump.

Resultaten från ANOVA-testet sparades i en textfil (*ANOVA_Resultat.txt*) för dokumentation. Alla fem mått visade p-värden långt under 0.05, vilket bekräftar att

skillnaderna mellan versionerna är statistiskt signifikanta och inte slumpmässiga variationer.

5.7 Litteraturstudie

För att sätta detta arbete i ett vetenskapligt sammanhang och ge framtida studenter möjlighet att bygga vidare på denna studie genomfördes en litteraturstudie av relevanta källor inom JavaScript-prestanda, React och webbprestanda.

Följande böcker användes som primära källor:

Ackermann, P. (2024) JavaScript: Mastering JavaScript From Basics to Advanced Topics. Inc Publishing.

Denna bok gav en omfattande grund i modern JavaScript-utveckling, inklusive kapitel om DOM-manipulation, asynkron programmering och moderna web APIs som är relevanta för prestanda jämförelsen.

O'Reilly Media (2024) Fluent React. O'Reilly Media, Inc.

Denna nyttigvna bok gav djup förståelse för React's arkitektur, Virtual DOM-konceptet och moderna React-patterns som hooks och funktionella komponenter.

Följande officiella dokumentation användes:

Google Developers (2020) Web Vitals. Mountain View: Google LLC.

Google's officiella dokumentation för Core Web Vitals gav den metodologiska grunden för vilka prestandamått som skulle mätas och varför dessa är viktiga för användarupplevelse.

Vetenskapliga artiklar:

Utöver böcker och dokumentation användes 10 vetenskapliga artiklar för att förstå tidigare forskning inom området (se Referenser-sektionen för fullständig lista). Särskilt relevanta var Andersson (2020), Cechinel, Laranjeira & Figueiredo (2023), Siahaan & Vianto (2022) och Van Riet et al. (2023).

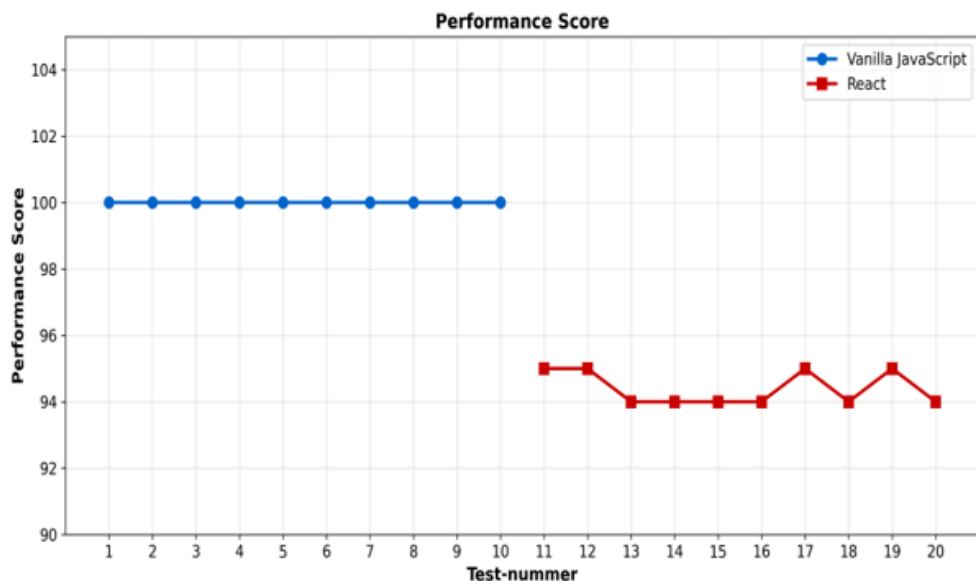
För framtida studenter som vill bygga vidare på detta arbete rekommenderas Ackermann (2024) för JavaScript-grund, Fluent React (2024) för React-förståelse och Google's Core Web Vitals-dokumentation för prestanda metodik.

6 Utvärdering

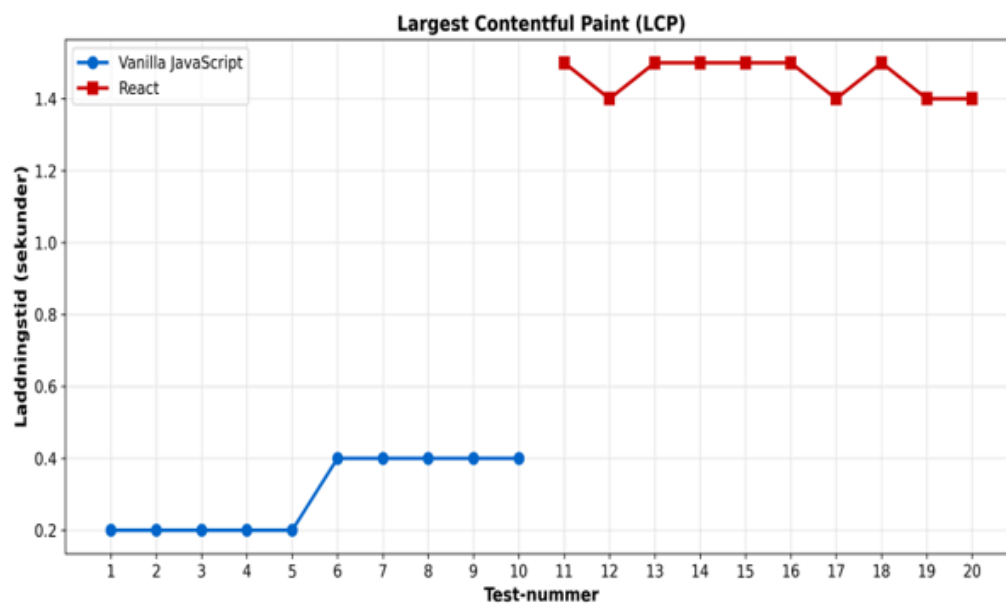
Detta kapitel presenterar resultaten från prestandatestningen av Vanilla JavaScript och React-versionerna av produktkatalogen. Först presenteras de uppmätta värdena från Lighthouse-testerna, följt av en statistisk analys av skillnaderna mellan versionerna.

6.1.1 Visuell presentation av testresultat

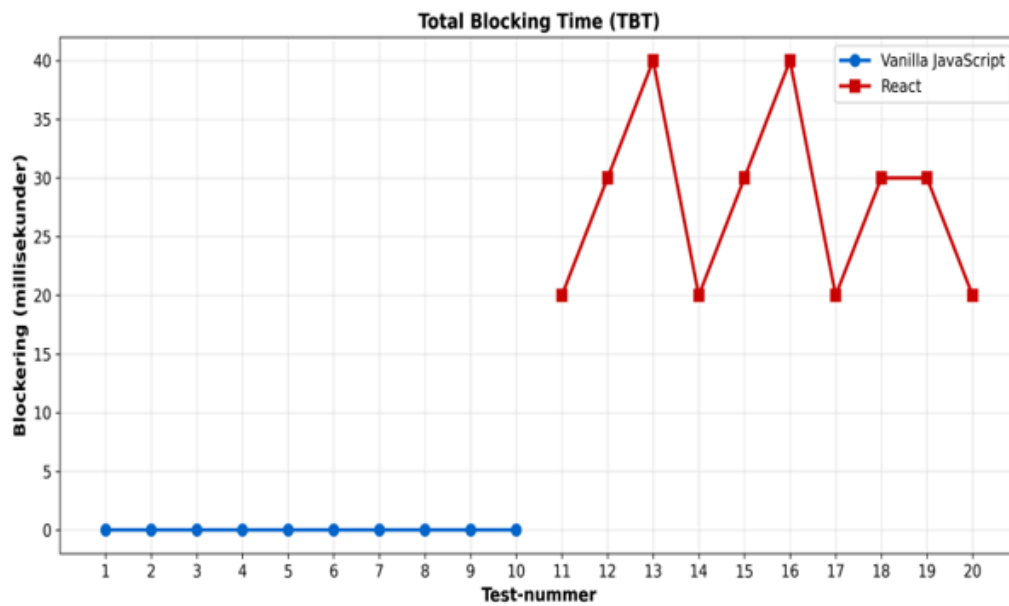
Figur 1-5 visar linjediagram för samtliga fem prestandamått över alla 20 tester.



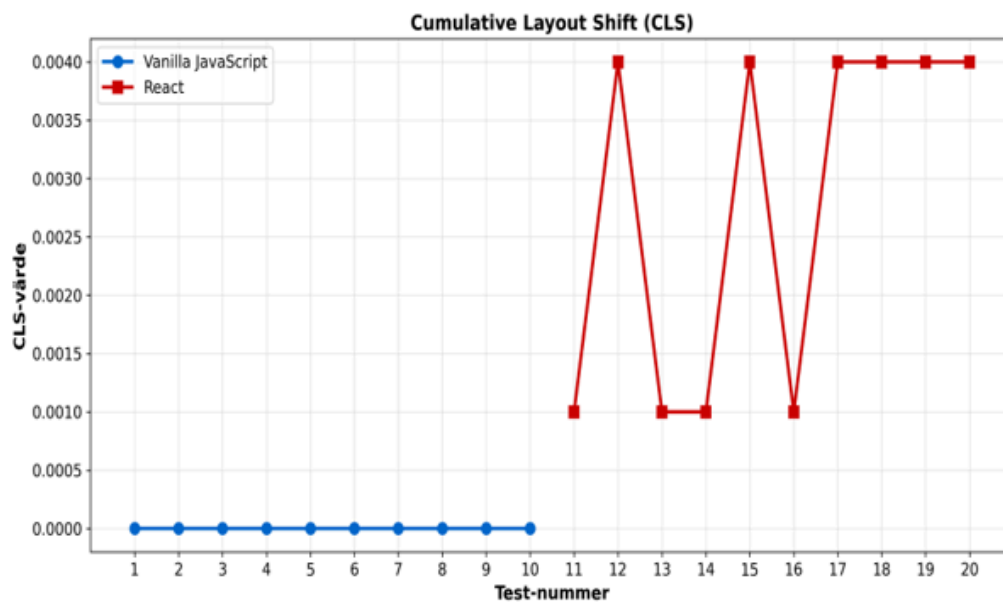
Figur 1: Performance Score visar att Vanilla JavaScript uppnådde perfekt poäng (100) i alla tester medan React varierade mellan 94-95.



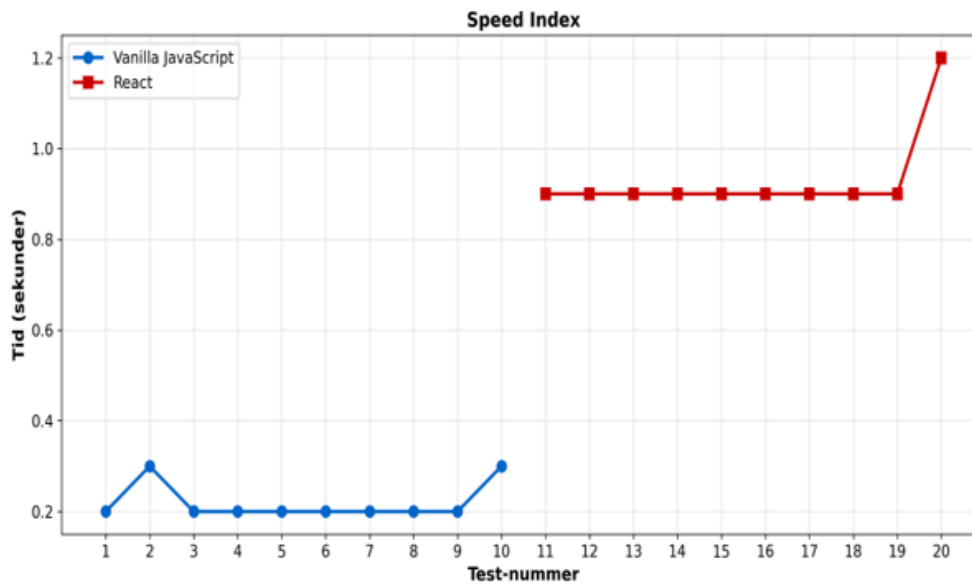
Figur 2: LCP (Largest Contentful Paint) visar tydligt att Vanilla JavaScript laddade på 0.2-0.4 sekunder medan React tog 1.4-1.5 sekunder. Nästan 5 gånger skillnad.



Figur 3: TBT (Total Blocking Time) visar att Vanilla JavaScript hade ingen blockering (oms) medan React varierade mellan 20-40ms.



Figur 4: CLS (Cumulative Layout Shift) visar minimal skillnad, båda versionerna hade värden nära noll, fast man ser ändå tydligt att Vanilla JavaScript presterade bättre.



Figur 5: Speed Index visar att Vanilla JavaScript renderade på 0.2-0.3 sekunder medan React tog 0.9-1.2 sekunder.

6.1.2 Presentation av undersökning

Alla tester genomfördes med Google Lighthouse i Chrome DevTools Desktop-läge. Varje test mätte fem huvudsakliga prestandamått som är relevanta för användarupplevelse och Core Web Vitals (Google Developers 2020):

- **Performance Score** - Ett övergripande prestanda-betyg på en skala 0-100 där 90-100 anses som utmärkt.
- **Largest Contentful Paint (LCP)** - Mäter laddningstid genom att registrera när det största synliga elementet renderas. Värden under 2,5 sekunder klassas som bra.
- **Total Blocking Time (TBT)** - Mäter hur länge sidan är blockerad från användarinteraktion under laddning. Lägre värden är bättre.
- **Cumulative Layout Shift (CLS)** - Mäter visuell stabilitet genom att spåra oväntade layoutförändringar. Värden under 0,1 anses som bra.
- **Speed Index** - Mäter hur snabbt innehållet visas visuellt under laddning. Lägre värden indikerar snabbare upplevd laddning.

Tabell 1 visar en sammanfattning av medelvärden för båda versionerna samt den statistiska analysen.

Mått	Vanilla JS	React	Skillnad	P-värde
Performance Score	100.00	94.40	-5.60	< 0.000001
LCP (s)	0.30	1.46	+1.16	< 0.000001
TBT (ms)	0.0	28.0	+28.0	< 0.000001
CLS	0.0000	0.0028	+0.0028	0.00002
Speed Index (s)	0.22	0.93	+0.71	< 0.000001

Tabell 1: Sammanfattning av prestandamätningar (medelvärden från 10 tester per version)

Som tabellen visar, presterar Vanilla JavaScript bättre än React på samtliga mått. De negativa värdena i kolumnen 'Skillnad' indikerar att Vanilla JavaScript är snabbare (lägre LCP och Speed Index) eller har mindre overhead (lägre TBT och CLS). Performance Score för Vanilla JavaScript var perfekt 100 i alla tester medan React varierade mellan 94-95.

Vanilla JavaScript-resultat

Vanilla JavaScript-versionen testades genom att öppna *index.html* med VS Code Live Server på port 5500. Totalt genomfördes 10 tester där varje test föregicks av en fullständig omstart av Chrome för att säkerställa rena förhållanden.

Resultaten från Vanilla JavaScript-versionens tester visas i Tabell 2. Värdena är remarkabelt konsekventa över alla tester, vilket indikerar hög reproducerbarhet och stabilitet i mätningarna.

Test	Performance	LCP (s)	TBT (ms)	CLS	Speed Index
1	100	0.2	0	0	0.2
2	100	0.2	0	0	0.3
3	100	0.2	0	0	0.2
4	100	0.2	0	0	0.2
5	100	0.2	0	0	0.2
6	100	0.4	0	0	0.2
7	100	0.4	0	0	0.2
8	100	0.4	0	0	0.2
9	100	0.4	0	0	0.2
10	100	0.4	0	0	0.3

Tabell 2: Detaljerade testresultat för Vanilla JavaScript (Test 1-10)

Vanilla JavaScript uppnådde perfekt Performance Score (100/100) i samtliga tio tester. LCP varierade minimalt mellan 0.2s och 0.4s, vilket båda ligger väl under Googles rekommenderade gräns på 2.5s (Google Developers 2020). TBT och CLS var genomgående noll, vilket indikerar att sidan inte blockerar användarinteraktion under laddning och inte har några "layoutförskjutningar". Speed Index låg konstant på 0.2-0.3s, vilket klassificeras som utmärkt.

React-resultat

React-versionen testades genom att starta Vite development server med *npm run dev* på port 5173. Samma testprotokoll följs som för Vanilla JavaScript-versionen med fullständig omstart av Chrome mellan varje test.

Tabell 3 visar resultaten från React-versionens tio tester. Till skillnad från Vanilla JavaScript visade React viss variation mellan testerna, särskilt i Performance Score och TBT.

Test	Performance	LCP (s)	TBT (ms)	CLS	Speed Index
11	95	1.5	20	0.001	0.9
12	95	1.4	30	0.004	0.9
13	94	1.5	40	0.001	0.9
14	94	1.5	20	0.001	0.9
15	94	1.5	30	0.004	0.9
16	94	1.5	40	0.001	0.9
17	95	1.4	20	0.004	0.9
18	94	1.5	30	0.004	0.9
19	95	1.4	30	0.004	0.9
20	94	1.4	20	0.004	1.2

Tabell 3: Detaljerade testresultat för React (Test 11-20)

React uppnådde Performance Score mellan 94-95, vilket fortfarande klassificeras som utmärkt men lägre än Vanilla JavaScript. LCP varierade mellan 1.4s och 1.5s - fortfarande under Googles 2.5s-gräns men betydligt högre än Vanilla JavaScript. TBT varierade mellan 20-40ms vilket indikerar viss blockering under laddning. CLS varierade minimalt mellan 0.001-0.004, vilket fortfarande klassas som utmärkt. Speed Index låg konsekvent på 0.9s med ett undantag på 1.2s.

6.2 Analys

För att statistiskt validera skillnaderna mellan versionerna genomfördes ett ANOVA-test för varje prestandamått. ANOVA-testet (Analysis of Variance) jämför variansen mellan två eller fler grupper för att avgöra om skillnaderna är statistiskt signifikanta eller om de kan förklaras av slumpmässig variation. I vetenskaplig forskning används vanligen en signifikansnivå på $p < 0.05$, vilket betyder att om p-värdet är lägre än 0.05 kan vi med 95% säkerhet säga att skillnaden inte beror på slumpen utan är en verklig skillnad mellan grupperna. Alla fem mått som testades visade p-värden långt under denna gräns, vilket bekräftar att skillnaderna mellan Vanilla JavaScript och React är verkliga och reproducerbara.

ANOVA-analysen utfördes med Python och scipy.stats-biblioteket. Resultaten visas i Tabell 4.

Mått	F-statistik	P-värde	Signifikant?
Performance Score	1176.0000	< 0.000001	Ja
LCP	976.6452	< 0.000001	Ja
TBT	126.0000	0.0000000015	Ja
CLS	32.6667	0.0000202877	Ja
Speed Index	467.7216	< 0.000001	Ja

Tabell 4: ANOVA-resultat för samtliga prestandamått

Resultaten från ANOVA-analysen är entydiga. Samtliga fem prestandamått visar p-värden långt under 0.05, vilket innebär att skillnaderna mellan Vanilla JavaScript och React är statistiskt signifikanta. Detta bekräftar att de observerade skillnaderna inte beror på slumpmässig variation utan är verkliga skillnader mellan versionerna.

De höga F-statistikvärdena (särskilt för Performance Score och LCP) indikerar att variansen mellan grupperna är mycket större än variansen inom grupperna. Detta stärker ytterligare slutsatsen att skillnaderna är substantiella och konsekventa.

Baserat på dessa resultat kan nollhypotesen (H_0) förkastas och hypotesen (H_1) accepteras: Vanilla JavaScript ger statistiskt signifikant bättre prestanda än React för denna typ av produktkatalog-applikation.

6.3 Slutsatser

Denna pilotstudie har gett värdefull insikt i prestanda skillnaderna mellan React och Vanilla JavaScript för en produktkatalog-applikation med 500 produkter. Resultaten visar tydligt att Vanilla JavaScript presterar signifikant bättre än React på samtliga uppmätta Core Web Vitals-mått. Pilot Studiens främsta syfte var dock inte att dra definitiva slutsatser, utan att identifiera lämpliga mätmetoder och parametrar för en mer omfattande huvudstudie.

6.3.1 Vad pilotstudien har lärt oss

Metodvalidering

Pilotstudien bekräftade att Google Lighthouse är ett lämpligt verktyg för att mäta prestandaskillnader mellan JavaScript-implementationer. De konsekventa resultaten över 10 tester per version visar att mätmetoden är tillförlitlig och reproducerbar.

Identifierade prestandaskillnader

De fem Core Web Vitals-måtten (Performance Score, LCP, TBT, CLS, Speed Index) visade alla statistiskt signifikanta skillnader mellan versionerna. LCP och Speed Index visade störst skillnader, vilket tyder på att dessa mått är särskilt viktiga för framtida mätningar.

6.3.2 Begränsningar i pilotstudien

Pilotstudien hade flera begränsningar som behöver adresseras i huvudstudien:

- **Begränsat produktantal:** endast 500 produkter testades. React's Virtual DOM kan potentiellt visa större fördelar vid mycket större dataset.
- **Enhets Begränsning:** Alla tester genomfördes på desktop. Prestandaskillnader kan vara annorlunda på mobila enheter.
- **Nätverks Begränsning:** Testerna kördes lokalt. Verkliga användare har ofta långsammare uppkopplingar.
- **Användarbeteende:** Testerna mätte endast initial laddning. Verkliga användare interagerar upprepat med applikationen.

6.3.3 Parametrar att justera för huvudstudien

Baserat på pilot studiens resultat föreslås följande parametrar för de riktiga mätningarna:

Produktantal: Testa 100, 500, 1000, 2000 och 5000 produkter för att förstå hur prestanda skalas.

Enhets Variation: Testa på desktop (high/budget) och mobil (high/mid/budget).

Nätverksförhållanden: Simulera 3G, 4G, Wi-Fi och localhost.

Interaktion Mätning: Mät prestanda vid upprepad sökning, filtrering och sortering.

6.3.4 Sammanfattning

Pilotstudien har framgångsrikt validerat mätmetoden och identifierat tydliga prestandaskillnader mellan React och Vanilla JavaScript för en produktkatalog med 500 produkter. För att dra mer generaliserbara slutsatser behöver huvudstudien utökas med fler produktantal, olika enheter, olika nätverksförhållanden och användarinteraktionmätningar. Pilot Studiens främsta bidrag är att den har etablerat en stabil metodologisk grund för dessa utökade mätningar.

7 Avslutande diskussion

7.1 Diskussion

Resultaten från denna studie ger viktiga insikter om prestandaskillnader mellan React och Vanilla JavaScript som är relevanta för både praktiserande webbutvecklare och den bredare diskussionen om när ramverk är motiverade.

7.1.1 Tolkning av resultat

Den mest slående skillnaden mellan versionerna var i LCP (Largest Contentful Paint), där React var nästan fem gånger långsammare än Vanilla JavaScript. Detta kan förklaras av att React måste ladda sitt bibliotek, skapa Virtual DOM och sedan rendera till riktig DOM, medan Vanilla JavaScript direkt manipulerar DOM utan något mellansteg. Som Cechinel, Laranjeira och Figueiredo (2023) påpekade kan ramverks overhead ha betydande påverkan på prestanda, särskilt vid initial rendering.

Även Total Blocking Time (TBT) visade skillnader. React hade genomsnittligt 28ms blockering jämfört med Vanilla JavaScript's oms. Detta tyder på att Reacts initialiseringsprocess blockerar huvudtråden under laddning, vilket kan påverka användarupplevelsen negativt. Van Riet et al. (2023) visade i sin industristudie att sådana objektiva mätningar har stark koppling till hur användare subjektivt upplever laddningstiden.

Att Vanilla JavaScript uppnådde perfekt Performance Score (100/100) i samtliga tester medan React varierade mellan 94-95 visar också på högre konsistens. Detta kan bero på att Vanilla JavaScript har färre rörliga delar och färre faktorer som kan påverka prestandan mellan körningar.

7.1.2 Jämförelse med tidigare forskning

Resultaten stämmer väl överens med Anderssons (2020) studie som också fann att Vanilla JavaScript har betydande fördelar när det gäller initial laddningstid och minnesanvändning. Andersson testade dock inte specifikt Core Web Vitals utan fokuserade mer på DOM-manipulation och filstorlek. Den här studien kompletterar därför Anderssons arbete genom att använda moderna mätstandarder som är direkt kopplade till användarupplevelse och SEO-ranking.

Siahaan och Vianto (2022) jämförde olika JavaScript-ramverk med Lighthouse och fann att prestanda varierar beroende på applikationstyp. Deras studie inkluderade inte Vanilla JavaScript, men de betonade vikten av att välja teknik baserat på specifika användningsfall snarare än allmän popularitet. Detta stöds av resultaten i denna studie - för en relativt enkel produktkatalog verkar Vanilla JavaScript vara det bättre valet.

Cechinel, Laranjeira och Figueiredo (2023) visade att React ofta kräver mer minne men kan vara snabbare vid komplexa uppdateringar tack vare Virtual DOM. Detta är en viktig nyans - även om denna studie visar att Vanilla JavaScript är snabbare för initial laddning, kan React fortfarande ha fördelar för applikationer med mycket dynamiska uppdateringar. För en produktkatalog där användaren främst söker, filtrerar och sorterar relativt statisk data verkar dock Reacts Virtual DOM inte ge några märkbara fördelar.

7.1.3 Praktiska implikationer

Resultaten har flera praktiska implikationer för webbutvecklare. För enklare webbapplikationer som produktkataloger, portfolios eller informationssidor kan Vanilla JavaScript vara ett bättre val än React om prestanda är prioriterat. Som Selakovic och Pradel (2016) visade är ineffektiv användning av ramverk en vanlig orsak till prestandaproblem, och att välja bort ett ramverk helt kan vara den mest effektiva optimeringen.

Samtidigt är det viktigt att betona att prestanda inte är det enda som spelar roll vid val av teknik. React kan erbjuda fördelar som bättre kodorganisation, återanvändbarhet av komponenter och ett större ekosystem av bibliotek och verktyg. För team som redan är bekväma med React kan produktivitetsvinster uppväga prestandaförlusterna, särskilt om applikationen växer i komplexitet över tid.

Core Web Vitals påverkar också SEO-ranking (Google Developers 2020), vilket gör prestanda extra viktigt för webbplatser som är beroende av organisk trafik. En förbättring av LCP från 1.5s till 0.3s kan ha konkret påverkan på både användarupplevelse och synlighet i sökmotorer.

7.1.4 Begränsningar och validitet

Studien har flera begränsningar som är viktiga att beakta. För det första testades bara en specifik typ av applikation (produktkatalog med 500 produkter). Resultaten kan inte generaliseras till alla typer av webbapplikationer. Som Siahaan och Vianto (2022) påpekade presterar olika tekniker olika beroende på användningsfall. Applikationer med mycket komplexa användargränssnitt, realtidsuppdateringar eller avancerad state-hantering kan visa helt andra resultat.

För det andra testades bara med 500 produkter. Det vore intressant att se hur prestandan skalas med 50, 5000 eller 50000 produkter. Möjligen skulle React's Virtual DOM visa större fördelar vid mycket större dataset där selektiva uppdateringar blir mer kritiska.

För det tredje genomfördes testerna i en kontrollerad miljö. Som Biørn-Hansen et al. (2020) noterade kan riktiga användare ha olika enheter, internetuppkopplingar och webbläsare som kan påverka resultaten annorlunda. En användarstudie hade gett värdefull kompletterande data om upplevd prestanda.

För det fjärde användes development-versioner av både Vanilla JavaScript och React under testningen. En produktionsbyggd React-app med minifiering och optimering hade troligen presterat bättre. Dock användes också en enkel utvecklingsserver för Vanilla JavaScript, så förhållandet mellan versionerna bör vara relativt rättvist.

Trots dessa begränsningar ger studien värdefulla insikter. De höga F-statistikvärdena och extremt låga p-värdena från ANOVA-testet (alla < 0.05 , många < 0.000001) indikerar att skillnaderna är robusta och inte beror på slumpmässig variation. De 20 upprepade mätningarna ger också god tillförlitlighet enligt principerna från Selakovic och Pradel (2016) om vikten av upprepade mätningar i JavaScript-prestandastudier.

7.2 Slutsats

Denna studie visar tydligt att för produktkatalog-applikationer med 500 produkter presterar Vanilla JavaScript statistiskt signifikant bättre än React när det gäller Core Web Vitals. Skillnaderna är inte marginella - Vanilla JavaScript laddade nästan fem gånger snabbare (LCP) och hade konsekvent perfekt Performance Score. För utvecklare som bygger liknande applikationer där prestanda är kritiskt kan Vanilla JavaScript vara ett bättre val än React, särskilt när applikationens komplexitet inte motiverar ett ramverks overhead.

Samtidigt är det viktigt att betona att detta inte betyder att Vanilla JavaScript alltid är bättre än React. Val av teknik måste baseras på projektets specifika krav, teamets kompetens och långsiktiga underhållbarhet. Denna studie bidrar till kunskapsbasen genom att ge konkreta prestandadata för ett specifikt användningsfall, vilket kan hjälpa utvecklare att göra mer informerade beslut.

7.3 Etiska aspekter

Studien involverar inga personuppgifter eller användare, vilket betyder att det inte finns några direkta etiska problem. Alla mätningar är tekniska och objektiva. Enligt Vetenskapsrådet krävs ingen etikprövning för den här typen av studie eftersom inga människor är inblandade.

För att säkerställa kvalitet kommer all kod att dokumenteras noggrant så att andra kan granska och upprepa studien. Det inkluderar exakta versioner av alla verktyg och beskrivning av vilken dator och testmiljö jag använder.

7.4 Samhällelig nytta

Arbetet kan bidra till att webbutvecklare gör bättre val av teknik, vilket kan leda till snabbare webbplatser och bättre användarupplevelse. Som Van Riet et al. (2023) visade påverkar bättre prestanda direkt användarnas upplevelse. Det kan också leda till mer energieffektiva webbapplikationer, vilket är positivt för miljön. Malavolta et al. (2017) visade att prestandaoptimering kan minska energiförbrukning, och snabbare webbsidor använder mindre energi både på servrar och på användarnas enheter.

7.5 Framtida arbete

I framtiden skulle det vara intressant att utöka studien till att inkludera fler ramverk som Vue.js och Svelte, samt testa med olika antal produkter för att se hur prestanda skalas. Det skulle också vara värdefullt att göra en användarstudie för att se hur riktiga användare upplever skillnaderna, likt Van Riet et al. (2023). En annan intressant fortsättning vore att testa med olika typer av applikationer, till exempel dashboards eller sociala medier, för att se om resultaten håller även där.

8 Referenser

Ackermann, P. (2024) JavaScript: Mastering JavaScript From Basics to Advanced Topics. Inc Publishing.

Andersson, L. (2020) 'JavaScript DOM Manipulation Performance: Comparing Vanilla JavaScript with Angular, React and Vue.js', Masteruppsats. Linnéuniversitetet.

Biørn-Hansen, A., Rieger, C., Grønli, T., Majchrzak, T.A. & Ghinea, G. (2020) 'An empirical investigation of performance overhead in cross-platform mobile development frameworks', Empirical Software Engineering, 25, s. 2997-3040.

Cechinel, A., Laranjeira, M. & Figueiredo, R. (2023) 'Examining the Performance Implications of Design Patterns in Front-end Web Development: A Preliminary Comparative Study with React and Vue.js', Proceedings of the 29th Brazilian Symposium on Multimedia and the Web. Rio de Janeiro: ACM, s. 145-154.

Google Developers (2020) Web Vitals. Mountain View: Google LLC.

Kaluža, M., Trošelj, K. & Vukelić, B. (2018) 'Comparison of Front-End Frameworks for Web Applications', Zbornik Veleučilišta u Rijeci, 6(1), s. 261-282.

Malavolta, I., Procaccianti, G., Noorland, P. & Vukmirovic, P. (2017) 'Assessing the Impact of Service Workers on the Energy Efficiency of Progressive Web Apps', Proceedings of the IEEE/ACM 4th International Conference on Mobile Software Engineering and Systems. Buenos Aires: IEEE/ACM, s. 35-45.

Oliveira, W., Moraes, B., Castor, F. & Fernandes, J.P. (2023) 'Analyzing the Resource Usage Overhead of Mobile App Development Frameworks', Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE). Oulu: ACM.

O'Reilly Media (2024) Fluent React. O'Reilly Media, Inc.

Selakovic, M. & Pradel, M. (2016) 'Performance Issues and Optimizations in JavaScript: An Empirical Study', Proceedings of the 38th International Conference on Software Engineering. Austin: IEEE/ACM, s. 61-72.

Siahaan, M. & Vianto, V.O. (2022) 'Comparative Analysis Study of Front-End JavaScript Frameworks Performance Using Lighthouse Tool', Jurnal Mantik, 6(3), s. 2462-2468.

Van Riet, N., Papadopoulos, D., Driesen, K. & De Turck, F. (2023) 'Optimise along the way: An industrial case study on web performance', Journal of Systems and Software, 201, s. 111671.

